

No Measurable Benefit from Verifier-Assisted Generate→Validate→Repair under Shared-Initial Generation: A Preregistered Emulation Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and autonomously executed a paired emulation study (study-cnd-2026-001) to evaluate whether a verifier-assisted generate→validate→repair pipeline increases oracle pass rates for LLM-generated network configurations in emulated NetOps intent→configuration tasks. The frozen design used shared_initial_candidate generation semantics, declared model gpt-5-mini, a deterministic validator (deterministic_netops_validator_v5), and a repair budget ≤1 call/task. Execution produced 200 complete paired units (400 episodes); model_calls_used=200 and repair calls invoked=0. Results: baseline 200/200 oracle passes; guarded 200/200 oracle passes; absolute paired difference 0.00 (paired bootstrap 95% CI [0.00,0.00], bootstrap_resamples=10000, seed=1729); McNemar two-sided p=1.00. The null outcome is design-bound: the preregistered shared_initial_candidate semantics delivered identical passing candidates to both arms and never exercised repair. We release the full audited artifact bundle (preregistration, prompts, raw outputs, provider traces, validator traces, execution manifest and SHA-256 digests) and give artifact-grounded recommendations for follow-ups (independent generation semantics, calibrated difficulty, larger repair budgets, validator stress tests).

I. INTRODUCTION

Generative LLMs are being explored as intent→configuration agents for network operations because they can map natural-language intent into vendor CLI sequences and programmatic device changes. Operational safety requires objective guarantees (e.g., transient reachability, protected interfaces, "make-before-break" semantics) that are not naturally enforced by unconstrained generation. A prevalent architectural response is generate→validate→repair: a generator proposes a candidate configuration, a deterministic validator (oracle) verifies required safety and policy predicates in an emulated model, and a repair module attempts corrective generation if the oracle fails. Prior work reports encouraging gains using constraint-guided generation or verifier-guided repair, but reproducible, preregistered, artifact-anchored comparisons are scarce.

We preregistered study-cnd-2026-001 to provide a confirmatory, fully archived measurement of verifier assistance under a precisely specified experimental design. The preregistration froze the estimand (paired_success_rate_difference_guarded_minus_baseline), the task bank stratification (Npairs=200), the generator semantics (shared_initial_candidate), the declared model (gpt-5-mini), the deterministic validator

(deterministic_netops_validator_v5), and the analysis executor (paired_binary_analysis_v1). All raw outputs, provider traces, validator traces, scoring records, and per-file SHA-256 digests were archived for deterministic audit. The principal research question was: Does the verifier-assisted generate→validate→repair pipeline increase oracle pass rate relative to unconstrained generation across the preregistered 200 tasks? Our contribution is an artifact-grounded account of the preregistered run, exact quantitative reporting, deterministic reconciliation of artifacts and hashes, explanation of why the run produced a design-bound null result, and concrete artifact-anchored recommendations for informative follow-ups.

II. RELATED WORK

Work on LLMs for NetOps spans intent translation, constrained generation, verifier-augmented pipelines, and repair loops. Representative artifacts motivating our design include MeshAgent (constraint-guided generation for network management), NetConfEval (intent→configuration benchmarks), and verifier-grounded program-repair workflows (StruSpec) that emphasize executing structured validators and returning counterexamples for repair. Recent surveys of LLMs for networking and AIOps contextualize the rising use of validators and RAG methods for operational grounding (see cited_record_ids). Unlike many prior reports, our run was preregistered before execution, archived with deterministic hashes, and executed under frozen generation semantics; we therefore can attribute the observed null result to an explicit, auditable design choice (shared_initial_candidate) rather than opaque post-hoc decisions.

III. METHODOLOGY

Preregistration and frozen plan (artifact): We preregistered study-cnd-2026-001 (preregistration SHA-256 a8969ac01f3b1bb7585872c58a92f72d0752160dd4e6e01cbafbf0d15073ad). The registration froze primary and secondary estimands, sampling plan, generation semantics = shared_initial_candidate, declared model = gpt-5-mini, validator = deterministic_netops_validator_v5, maximum_repair_calls_per_task = 1, analysis method (paired bootstrap percentile CI with 10,000 resamples, seed=1729) and stopping rules. The immutable initial master prompt that launched the autonomous post-lock

run is archived at provenance/master_prompt.txt (SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba) and is reported in the Disclosure Statement.

Task bank, stratification, and manifest (artifact): The pre-registered task bank comprised 200 paired tasks stratified by protocol: BGP/OSPF/ACL/VLAN = 50 pairs each, with vendor allocation per protocol Cisco=17, Juniper=17, Arista=16 (total Cisco=68, Juniper=68, Arista=64) and complexity simple=100 / complex=100. Every task entry (intent, initial_state, expected_final_state, required_sequence, constraints) was recorded in execution/task_manifest.jsonl; representative entries/validator traces are archived (e.g., execution/scoring/task-000001-baseline.json; SHA-256 eaf96aec12c30860...). Table 1 in the artifacts (analysis/condition_summary.csv SHA-256 1cb072b4...) enumerates the task bank summary.

Generation semantics and model configuration (artifact): The frozen generation semantics = shared_initial_candidate produced exactly one generator call per pair; that single generated candidate was supplied identically to both baseline and guarded arms. This choice intentionally reduced model calls (model_calls_used=200) and is recorded in execution/model_configuration.json (SHA-256 1acce92558edeb13...). The declared generator was gpt-5-mini via the hosted_netops_gvr_v1 adapter; all provider calls and their traces are archived under execution/provider_calls/ with per-file SHA-256 digest entries in execution/execution_manifest.json. The generation adapter recorded cache/miss counts and call metadata; call cache was not reused across conditions (cache_hit_count=0, cache_miss_count=200).

Validator, repair budget, and scoring (artifact): We used deterministic_netops_validator_v5 as the objective oracle; each scoring artifact contains the validator trace (normalized_configuration, parsed_command_count, final_state, valid boolean, and violation list). Repair was permitted per pre-registration but in this run repair_applied=false for all pairs and repair traces were therefore absent. Scoring artifacts for each episode are archived in execution/scoring/*.json (example: execution/scoring/task-000001-baseline.json SHA-256 eaf96a...). Scoring rule = full_intent_constraint_validation; scorer_version recorded in each file ensures reproducibility.

Execution, logging and immutability (artifacts): The autonomous run archived raw responses (execution/responses/*.txt), provider traces (execution/provider_calls/*.json), scoring records (execution/scoring/*.json), and the per-artifact SHA-256 manifest execution/execution_manifest.json. Deterministic reconciliation checks (analysis/deterministic_reconciliation.json SHA-256 799b23b9...) verify that every scoring record links to a stored raw response and provider trace, and that the complete_pair_count=200 equals the planned pairs. Analysis input and outputs are archived under analysis/ with SHA-256 digests (e.g., analysis/paired_contingency_table.csv SHA-256 7bc1bd2a...). Table 2 in the artifacts reports the confirmatory contingency used for inference.

IV. RESULTS

Execution and primary quantitative outcome (artifact-grounded): The run completed per preregistration: planned_episode_count=400, completed_episode_count=400, failed_episode_count=0, model_calls_used=200. The deterministic validator accepted every shared candidate: baseline_success_count = 200/200 (100%); guarded_success_count = 200/200 (100%). The paired contingency is n11=200, n10=0, n01=0, n00=0 (analysis/paired_contingency_table.csv SHA-256 7bc1bd2a...). The paired estimate for paired_success_rate_difference_guarded_minus_baseline = 0.00; paired bootstrap 95% percentile CI = [0.00,0.00] (bootstrap_resamples=10000, seed=1729); exact McNemar two-sided p = 1.00. These results are present in analysis/results.json; the analysis log records the bootstrap parameters (analysis/analysis_log.jsonl SHA-256 940b48e7...).

Why repair was never observed (design causal chain): Under the preregistered shared_initial_candidate semantics a single generator call per pair produced an identical candidate for baseline and guarded conditions. Because the deterministic validator accepted every candidate in the 200-pair task bank, the repair stage was never invoked (repair_calls_invoked = 0 across all validator traces). Validator traces (examples in execution/scoring/task-00000X-*.json) include explicit fields repair_applied=false and validation_after.valid=true; these artifacts demonstrate that the guarded branch had no opportunity to improve over baseline. The null statistical outcome is therefore a direct consequence of the frozen generation semantics plus the (empirical) generator behavior on these tasks.

Representative examples and diagnostics (artifact excerpts): Representative scoring files (execution/scoring/task-000001-baseline.json SHA-256 eaf96aec...) show normalized_configuration, parsed_command_count, final_state dictionaries, and valid=true; execution/responses/task-000001-shared-initial.txt stores the shared candidate (SHA-256 f8de4d6e...). The analysis bundle includes a sorted paired difference figure (analysis/paired_difference_figure.svg SHA-256 ae5b8e0c...) that visually confirms the ceiling (all pairs concordant successes). Deterministic_reconciliation.json verifies accounting consistency and provider call audit (hosted_model_call_event_count=200; cache_hit_count=0).

Secondary and sensitivity diagnostics: Missingness_summary artifact (analysis/missingness_summary.csv SHA-256 808b3c00...) shows complete_pairs=200 and zero failed episodes; there were no preregistered exclusions invoked. Contamination_summary (analysis/contamination_summary.csv SHA-256 389227f0...) contains no flagged contamination items. All of these diagnostics are archived for auditing and reproducibility.

V. DISCUSSION

Interpretation and scope: The preregistered experiment answers the frozen estimand under the frozen generation semantics: verifier assistance provided no measurable benefit

because the verifier and repair stage were never exercised. This is a valid empirical outcome for the declared estimand and the observed generator behavior, and it is supported by the full artifact trail. The result should not be read as a general negative verdict on verifier-assisted pipelines: rather, it highlights a critical design interaction—generation semantics and task difficulty determine whether repairs can act. When a single shared candidate per pair routinely passes the oracle, there is no opportunity for guarded repairs and the paired contrast is uninformative about repair efficacy.

Operational implications and concrete follow-ups (artifact-anchored): The archived bundle (execution/, analysis/, provenance/) removes ambiguity about the design that caused the ceiling effect and enables immediate reproducible follow-ups. Two primary, low-risk variants that are directly executable using the archived prompts and task seeds are: (1) independent_generation semantics — change generation_semantics to generate one candidate per condition per pair (expected model_calls $\approx$ 400) so baseline and guarded arms sample distinct generator randomness; repairs become exercisable when baseline samples fail the validator; (2) calibrated difficulty and adversarialization — augment execution/task_manifest.jsonl with higher-difficulty tasks (e.g., additional required_sequence constraints, paired MTU/atomic changes) or boundary cases so that a nontrivial fraction of shared candidates fail. Both variants preserve the validator and analysis executor and can be executed under the same adapter while producing a new audited manifest and per-file SHA-256 digests for deterministic reconciliation. We recommend also increasing maximum_repair_calls_per_task (≥ 2) to allow iterative repair loops if the goal is to measure repair convergence.

Threats to validity (artifact-grounded): Internal validity — the decisive threats are generation semantics and prompt sensitivity; both were preregistered and are logged in execution/model_configuration.json and provider call traces. Construct validity — the deterministic validator enforces emulation-based safety predicates but may omit some semantic failure modes; validator traces include violation lists and oracle_coverage diagnostics to surface potential blind spots. External validity — the run used one declared model (gpt-5-mini) and synthetic/emulated tasks; reproduction across other models or production device images is necessary before operational generalization. Conclusion validity — the paired design was executed as registered; the statistical inference reflects the degenerate contingency (no discordant pairs) and is the correct conclusion under the frozen design.

Reproducibility instructions (artifact paths and hashes): The complete artifact set and per-file SHA-256 digests are recorded in execution/execution_manifest.json (artifact bundle). Key artifacts referenced in this manuscript: preregistration (SHA-256 a8969ac01f3b...), initial master prompt provenance (provenance/master_prompt.txt SHA-256 1872df1e...), model configuration (execution/model_configuration.json SHA-256 1acce92558ed...), paired contingency (analysis/paired_contingency_table.csv SHA-256 7bc1bd2a...),

condition summary (analysis/condition_summary.csv SHA-256 1cb072b4...), and representative scoring traces (execution/scoring/task-000001-baseline.json SHA-256 eaf96aec12c3...). To rerun a variant, prepare a new preregistration that modifies generation_semantics or repair budget, then execute the adapter; the framework will produce a new execution_manifest.json and deterministic_reconciliation.json for audit.

Artifacts released: All raw responses, provider traces, scoring JSON, task manifest, execution manifest, analysis artifacts, and deterministic reconciliation files are included in the released bundle (paths and SHA-256 digests appear in execution/execution_manifest.json) to permit exact replication and forensic review.

VI. LIMITATIONS

- Confirmatory conclusions apply only to the exact preregistered design (shared_initial_candidate semantics, single declared model gpt-5-mini, deterministic_netops_validator_v5, synthetic/emulated task bank of 200).
- Shared_initial_candidate execution semantics created a ceiling effect: because the single shared generated candidate per pair passed the deterministic oracle for every task, the repair stage was never invoked (repair_calls_invoked = 0), preventing measurable guarded benefit in this run.
- Emulation with a deterministic validator and synthetic tasks does not capture full production heterogeneity (real vendor images, runtime nondeterminism, device-specific quirks).
- Single-model execution limits generality across model families and instruction-tuning variants.
- Repair budget was intentionally limited (≤ 1 call/task) by preregistration; richer iterative repair policies remain unexplored in this run.
- Oracle blind spots (uncovered semantic failure modes) remain possible; oracle-coverage diagnostics are archived and annotated in the artifact bundle.

VII. CONCLUSION

Under the exact preregistered design (shared_initial_candidate semantics, declared model gpt-5-mini, deterministic validator, repair budget ≤ 1), verifier assistance produced no empirical improvement because the single shared generated candidate per pair passed the oracle in every case and repairs were never invoked (repair_calls_invoked=0). The study demonstrates the necessity of aligning generation semantics and task difficulty to the scientific estimand: measuring repair effectiveness requires generator variance or tasks that elicit validator failures. We publish the full audited artifact bundle so future runs can modify the generation semantics or task difficulty while preserving deterministic auditability.

DISCLOSURE STATEMENT

Mandatory	Disclosure	Statement	(artifact-backed):
- Initial	master	prompt	reference (immutable
archive):	provenance/master_prompt.txt		(SHA-256:

This master prompt launched the autonomous post-lock research run. The Research Director selected the broad topic family (Generative AI and LLMs for NetOps) pre-lock; all substantive research actions reported here were performed autonomously after the lock. - Preregistration (frozen): study id = study-cnd-2026-001; preregistration SHA-256 = a8969ac01f3b1bb7585872c58a92f72d0752160dd4e6e01cbafbfb The preregistration froze the primary hypothesis, sampling plan, generation semantics (shared_initial_candidate), declared model, validator, repair budget, stopping rules, and analysis executor. - LLM and adapter: declared model = gpt-5-mini used via hosted_netops_gvr_v1 adapter; model configuration artifact: execution/model_configuration.json (SHA-256 = 1acce92558edeb13cd97b75817329d332afe4a36d01d566f1a26

All provider call traces and raw responses are archived under `execution/provider_calls/` and `execution/responses/` with per-file SHA-256 digests recorded in `execution/execution_manifest.json`.

- Generation semantics (frozen): `shared_initial_candidate` (one generated candidate per pair and provided to both baseline and guarded conditions). This preregistered choice produced `model_calls_used=200` (one shared generation per 200 pairs) and is the proximate cause the guarded repair stage was never invoked (repair calls invoked = 0). The execution manifest records these counts and per-file hashes (`execution/execution_manifest.json`).
- Validator and oracle: `deterministic_netops_validator_v5` scored every candidate; representative scoring artifact: `execution/scoring/task-000001-baseline.json` (SHA-256 = `eaf96aec12c30860a6fd70987289cc71104134a3330b99c7e024f`).

All validator traces are archived and hashed. - Repair policy and budgeting: preregistered maximum_repair_calls_per_task = 1; in this run repair calls invoked = 0 (artifact-backed). Repair prompts would have been archived if used. - Analysis procedures (frozen): paired_binary_analysis_v1; paired bootstrap 10,000 resamples (seed=1729) and exact McNemar two-sided test. Confirmatory analysis artifacts: analysis/paired_contingency_table.csv (SHA-256 = 7bc1bd2a42b5ac3f7adb61db47cf8dbe0472f678032abcd1e7c44 and analysis/condition_summary.csv (SHA-256 = 1cb072bd4db7c8e29212ef28b97baccb66a507a7c8b8ceaad35b7e

- Execution and reproducibility: every prompt, model output, provider trace, validator trace, scoring record, manifest and analysis artifact is archived with SHA-256 digests. Deterministic reconciliation confirms arithmetic and accounting consistency (analysis/deterministic_reconciliation.json; SHA-256 = 799b23b993fcc9c04fc4d0a375e975937bc11d9f8c6595d43dce9

- Human involvement: pre-lock collaborative development and approval of the master prompt were performed by the Research Director. No human performed manual substantive

editing of the manuscript after the autonomy lock. Pre-lock development artifacts were not used as empirical evidence unless reproduced under the frozen post-lock workflow. - Deviations: none affecting the primary confirmatory analysis; model identifier did not change mid-run; no pairs excluded; 263/263 trials not triggered. Full execution manifest and per-file SHA-256 digests are included in the artifact bundle for audit.

REFERENCES

- [1] Yajie Zhou, Kevin Hsieh, Sathiya Kumaran Mani, Srikanth Kandula, Zaoxing Liu, "MeshAgent: Enabling Reliable Network Management with Large Language Models", 2025, 10.1145/3771567
- [2] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, Marco Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?", 2024, 10.1145/3656296
- [3] Zhengyang Ding, "Executable but Wrong: Verifier-Grounded Contracts and Counterexamples for LLM-Generated Music Programs", 2026, 10.31224/7995